

Binary Search

Binary Search — A search algorithm that finds a target by repeatedly halving the search space.

e.g. Find 7 in [1,3,5,7,9] "check middle 5, go right, find 7."

★ Only works on SORTED arrays. Always verify before applying.

Algorithm Steps

Set $low = 0$, $high = n-1$



Find $mid = (low + high) / 2$



If $arr[mid] == target$ 'return mid



If $target > arr[mid]$ 'low = mid + 1

Algorithm Steps (contd.)

If $\text{target} < \text{arr}[\text{mid}]$ then $\text{high} = \text{mid} - 1$



Repeat until $\text{low} > \text{high}$

TIME COMPLEXITY

$O(\log n)$

Advantages

- Fast "eliminates half the array each step"
- $O(\log n)$ vs $O(n)$ for linear search
- Works well on large sorted datasets

IN SHORT

1. Only works on sorted arrays
2. Time complexity: $O(\log n)$
3. Space complexity: $O(1)$ for iterative

Implementation Details

The overflow bug

★ Computing mid as $(\text{low} + \text{high}) / 2$ overflows on large arrays in fixed-width integer languages. This bug shipped in `java.util.Arrays` for nine years.

SAFE MIDPOINT

$$\text{mid} = \text{low} + \frac{\text{high} - \text{low}}{2}$$

Every place an off-by-one hides

- Initialising high to n instead of $n-1$
- Using $\text{low} < \text{high}$ when the loop needs $\text{low} \leq \text{high}$
- Forgetting to move past mid , so the range never shrinks
- Returning low when the target is absent instead of -1
- Assuming the array is non-empty before the first read
- Comparing with $==$ on floating-point values
- Letting mid be computed before the emptiness check
- Rounding mid down when the recurrence assumes rounding up
- Mixing inclusive and exclusive bounds in the same function
- Shrinking by mid rather than $\text{mid} + 1$ on the right branch

Every place an off-by-one hides (contd.)

- Shrinking by mid rather than mid - 1 on the left branch
- Treating a signed length as unsigned
- Searching a rotated array with the unrotated comparison
- Using the wrong comparator for a descending sort
- Assuming duplicates resolve to the first match
- Assuming duplicates resolve to the last match
- Trusting that the input was actually sorted
- Recursing without a base case for the empty range
- Blowing the stack on a recursive version for huge inputs
- Reading `arr[mid]` after the bounds have already crossed
- Caching the length across a mutation of the array
- Applying it to a linked list and losing $O(\log n)$ entirely
- Reusing a stale mid from the previous iteration
- Writing the tests against the same wrong mental model

Invariant — A condition true before and after every loop iteration
“here, that the target lies within $[low, high]$ if it is present at all.

IN SHORT

1. Prefer $low + (high - low) / 2$
2. Pick one bound convention and never mix them
3. State the loop invariant before writing the loop

When Not To Use It

- Unsorted data " sorting first costs $O(n \log n)$
- Very small arrays, where linear scan wins on cache locality
- Linked lists, where random access is $O(n)$
- Data that changes between searches

Binary Search	Hash Lookup
$O(\log n)$	$O(1)$ average
Needs order	Needs a hash function
Supports range queries	Point lookups only

Interpolation search — A variant that guesses the position from the value's distance between the bounds, reaching $O(\log \log n)$ on uniformly distributed data.

e.g. Looking up a name in a phone book by opening near where you expect it.

IN SHORT

1. Sorted access is the price of admission
2. A hash map beats it for pure point lookups
3. Range queries are where it still wins